Rapport de stage

Enzo Deyrich

Étudiant : Enzo Deyrich

Période : 05 janvier au 27 février (2 mois)

Etablissement : Epsi Lille

Entreprise : SNCF e.solutions

2026 **SN2 BTS SI**

Remerciement

Avant d'entamer la lecture de ce rapport, je tiens à exprimer mes remerciements envers toutes les personnes qui ont contribué au bon déroulement de ce stage au sein de SNCF e.Solutions. Grâce à vous, ces deux mois ont été une expérience extrêmement enrichissante, tant sur le plan technique que sur le plan humain.

Je souhaite tout d'abord adresser mes plus sincères remerciements à Nicolas Dole, chef de l'équipe AS. Il m'a accordé sa confiance en m'accueillant au sein de son service pour ce stage. Je lui suis particulièrement reconnaissant pour son accueil chaleureux et pour avoir veillé à ma parfaite intégration dès les premiers jours. Je le remercie également pour tous les précieux conseils qu'il m'a prodigué au fil des semaines.

Un immense merci à mon tuteur de stage, Hervé Chapuis. Je tiens à souligner sa patience à toute épreuve et son dévouement pour me transmettre ses connaissances. Merci pour tout le temps qu'il a consacré pour moi, à m'expliquer en détail les concepts de développement afin de me faire monter en compétences. Son expertise technique a été indispensable pour me débloquer sur des sujets complexes de l'API Fabric. Merci enfin pour sa bienveillance et pour tous ses conseils qui me serviront tout au long de mon parcours.

Je tiens également à remercier Alexandre Gabet qui a pris le temps, pendant deux à trois jours, de me montrer une montée en version d'outils Data de pointe tels qu'Airflow et OpenMetadata. Il m'a appris des notions totalement nouvelles pour moi, et j'ai beaucoup apprécié ses conseils concernant mon avenir professionnel.

Mes remerciements s'adressent tout particulièrement à Youssef, avec qui j'ai collaboré au quotidien. Merci pour le temps précieux qu'il m'a accordé lors du démarrage de ma mission. Son aide m'a été indispensable pour la mise en place et la compréhension de mon environnement de développement.

Enfin, je souhaite remercier plus globalement l'ensemble de l'équipe AS qui m'ont intégré comme un membre à part entière lors des réunions du vendredi matin. Cette immersion dans un grand compte a définitivement confirmé mon attrait pour ce métier, et c'est en grande partie grâce à eux tous.

Sommaire

Rapport de stage Enzo Deyrich 1
Remerciement 1
Sommaire 2
Introduction 3
Présentation du projet 4
Intégration dans le projet 5
Objectifs fonctionnels 6
Enjeux pour l'entreprise 6
Etat initial du projet 7
Démarche suivie 8
Un suivi basé sur le compagnonnage technique 9
Organisation du travail et collaboration 9
Mise en place de l'environnement de développement 9
Organisation du projet 11
Planning 11
Compte-rendu et prise de notes 11
Réunions 11
Formation, documentation et premiers jours 12
Réalisation de la mission 12
Analyse et compréhension de la tâche 13
Phase de développement 14
Intégration et validation 16
Documentation et restitution du travail 17
Justification de la solution retenue 18
Comparatif logiciel et choix technologiques 19
Choix d'organisation et outils de gestion 20
Étapes essentielles 21
Fonctionnalités réalisées 21

Mise en place : outils et incidents 23
Évaluation des réalisations 23
Adéquation du travail 24
Réactions des demandeurs et intérêt pour l'entreprise 24
Propositions d'amélioration 25
La démarche RSE d'Ekeep-It 25
Engagement social 26
Engagement environnemental 26
Engagement économique local 26
Éthique professionnelle 26
Conclusion 26

Introduction

Dans la continuité de ma formation en informatique, j'ai réalisé mon stage de deuxième année, du 5 janvier au 27 février 2026, au sein de SNCF e.Solutions (entité de SNCF Connect & Tech), installée au cœur du pôle d'excellence numérique d'EuraTechnologies à Lille.

Si ma première expérience m'avait permis de découvrir le fonctionnement d'une ESN à taille humaine, ce second stage avait pour objectif de m'immerger dans une structure d'envergure industrielle. SNCF e.Solutions pilote des projets digitaux critiques utilisés par des millions d'usagers. Intégrer une telle organisation répondait à une volonté précise : sortir de ma zone de confort axée sur le développement web pur pour monter en compétence sur des sujets transverses comme la Data, le DevOps et l'automatisation à grande échelle.

J'ai intégré l'équipe AS (Architecture Solutions), un pôle d'expertise dédié à l'architecture Cloud. Ma mission principale a consisté à accompagner l'automatisation et la sécurisation des flux de données via la mise en place d'un portail pour les DataGateways.

Encadré par Hervé Chapuis, Architecte Cloud et expert Power BI, j'ai évolué dans un environnement technique exigeant et nouveau pour moi. Contrairement aux méthodes Agiles classiques, le suivi de ce stage mis en place autour d'un compagnonnage technique : un accompagnement étape par étape favorisant la montée en autonomie face aux blocages. J'ai ainsi pu manipuler des outils d'infrastructure et de CI/CD de référence tels que Jenkins, Ansible, GitLab et HashiCorp Vault, tout en collaborant via l'écosystème Microsoft (Teams, Outlook, VS Code).

Ce stage a été l'occasion idéale de découvrir de nouvelles technologies, de comprendre les rouages d'une architecture complexe et d'acquérir une première expérience significative dans l'automatisation de processus.

Présentation du projet

Le projet DataGateway s'inscrit dans la stratégie de modernisation de la gestion des données au sein de la SNCF. Dans un environnement où la donnée est un actif stratégique, les *datagateways* (passerelles techniques de données) jouent un rôle clé en assurant la connexion sécurisée entre les sources de données internes et les outils d'analyse ou de visualisation, comme Power BI. Ces passerelles sont essentielles pour fiabiliser les échanges d'informations tout en respectant les contraintes d'infrastructure de l'entreprise.

Pour faciliter leur exploitation, l'objectif du projet est de déployer un portail interne dédié, intégré à Automatio (la plateforme interne de la SNCF qui regroupe et unifie l'accès à divers cas d'usage). Dans ce contexte, le portail DataGateway a vocation à devenir le point d'entrée pour la gestion centralisée de ces passerelles. Depuis cette interface, les utilisateurs internes (équipes data, développeurs, administrateurs) pourront :

- Consulter l'état et les métadonnées des datagateways en temps réel.
- Créer, modifier ou supprimer des passerelles via des workflows sécurisés.
- Piloter des actions techniques spécifiques (par exemple, la mise hors service ou le renommage).

Mon stage au sein de l'équipe AS (Architecture Solutions) m'a permis de contribuer à ce projet dès sa phase de lancement, en participant à la conception et au développement des premières briques techniques nécessaires à cette intégration.

Intégration dans le projet

Le portail DataGateway ne part pas de zéro : il s'appuie sur un parc existant de datagateways déjà utilisées quotidiennement par les équipes métiers. Jusqu'à présent, la gestion de ces passerelles reposait principalement sur une combinaison d'interfaces natives fournies par les éditeurs, de scripts spécifiques créés à la volée, et de procédures manuelles souvent hétérogènes.

Le nouveau portail a pour objectif d'unifier ces pratiques en offrant une interface cohérente. Il ne s'agit pas d'effacer les outils existants, mais d'ajouter une couche d'ergonomie et d'automatisation indispensable pour :

- Simplifier les opérations courantes de maintenance et de déploiement.
- Réduire les risques d'erreurs liés aux interventions manuelles sur l'infrastructure.
- Standardiser les processus en les encapsulant dans des cas d'usage précis et documentés.

Objectifs fonctionnels

La création de ce portail répond à trois objectifs majeurs pour les équipes techniques et métiers :

1. Centraliser l'information : Rendre l'ensemble des datagateways visibles au même endroit. Le portail offre un accès unifié à la liste des passerelles et à leurs caractéristiques (nom, environnement, état, usages associés), mettant fin aux recherches dispersées entre différents outils.

2. Simplifier les opérations : Les actions de gestion (création, modification, suppression) seront exposées sous forme de boutons d'action. L'utilisateur déclenchera un scénario prédéfini qui se charge d'appeler les API nécessaires en arrière-plan. Par exemple, la mise hors service d'une passerelle se résumera à un clic : le système la renomme avec un préfixe « OFF_ » puis la désactive selon un processus.

3. Industrialiser les processus : En encapsulant ces actions dans des workflows automatisés, le portail réduit drastiquement la dépendance aux scripts manuels exécutés localement, garantissant des opérations répétables et sécurisées.

Enjeux pour l’entreprise

Pour un groupe de l'envergure de la SNCF, l'implémentation de ce portail répond à des enjeux stratégiques forts :

- Fiabilisation des échanges de données : Le bon fonctionnement des datagateways est crucial pour éviter des perturbations opérationnelles. Imposer des parcours standardisés via le portail renforce la sécurité et minimise les risques d’interruption de service.

- Optimisation de la productivité : En centralisant et en automatisant des tâches d'administration réseau et data souvent complexes, les équipes techniques gagnent un temps précieux et peuvent se concentrer sur des missions à plus forte valeur ajoutée.

- Gouvernance et maîtrise technique : Le portail simplifie l’application de règles communes de configuration et améliore la maîtrise des accès aux données, un enjeu crucial face aux réglementations strictes.

- Modernisation technologique : Ce projet permet de tirer parti des évolutions de l'écosystème Cloud et Data de la SNCF pour offrir un outil interconnecté, adapté aux défis de la gestion de données à grande échelle.

Etat initial du projet et améliorations à apporter

À mon arrivée dans l'équipe, les datagateways étaient opérationnelles, mais le portail en lui-même n'en était qu'à ses débuts.
Il y avait une première partie de fonction de connexion qui avait été codé en PowerShell par Herve Chapuis mon tuteur de stage.
Une documentation Microsoft concernant les API Fabric très dispersée et parfois incomplète, rendant la prise en main complexe.

Problématiques identifiées :

- Dette technique et hétérogénéité : Les requêtes et scripts d'administration n'étaient pas encore uniformisés ni intégrables facilement dans un backend moderne.

- Manque de documentation claire : L'interaction avec les API Fabric nécessitait un important travail de recherche et d'expérimentation à cause d'un manque de documentation de Microsoft.

- Risque d'erreur humaine : La manipulation manuelle des passerelles augmentait le risque de casser des flux de données en production.

Améliorations attendues et cadre de ma mission :

- Développement des briques fondatrices : Création des premiers scripts en Python (requêtes CRUD) pour dialoguer proprement avec les API et préparer le backend du futur portail.

- Automatisation : Traduction des actions manuelles complexes en cas d'usage simples pour l'utilisateur final.

- Mise en place de l'environnement : Configuration d'un environnement de développement complet et robuste (utilisation de Git, CI/CD avec Jenkins, etc.) pour industrialiser la suite du projet.

Démarche suivie

Contrairement à des projets scolaires classiques ou à ma précédente expérience, nous n'avons pas utilisé de méthode Agile (comme Scrum) pour la gestion de ce projet. Compte tenu de la complexité de l'environnement technique de la SNCF, le suivi de mon stage s'est plutôt articulé autour d'un compagnonnage technique de proximité avec mon tuteur, Hervé.

Un suivi basé sur le compagnonnage technique

L'attribution des tâches et le suivi ne se faisaient pas via un outil de tickets rigide, mais de manière beaucoup plus fluide. Mon tuteur de stage (Hervé Chapuis) me donnait mes missions soit lors de réunions Teams, soit en présentiel, ou encore par message.

Pour chaque nouvelle tâche, il prenait souvent deux à trois heures pour m'expliquer clairement les attentes, le contexte technique, et me lancer sur la bonne voie. Cette approche m'a permis de ne pas me sentir perdu face à l'ampleur de l'infrastructure. Ensuite, nous faisions un ou deux points par semaine pour voir mon avancement, ou de manière ponctuelle dès que je me retrouvais bloqué sur un problème.

En plus de mon tuteur, j'ai pu compter sur l'aide de l'équipe d'architectes, et tout particulièrement sur Youssef, un membre d'une autre équipe qui travaillait également sur le projet DataGateway.

Organisation du travail et collaboration

Au quotidien, notre outil principal de communication était Microsoft Teams, que ce soit pour des appels, des partages d'écran ou des messages rapides.

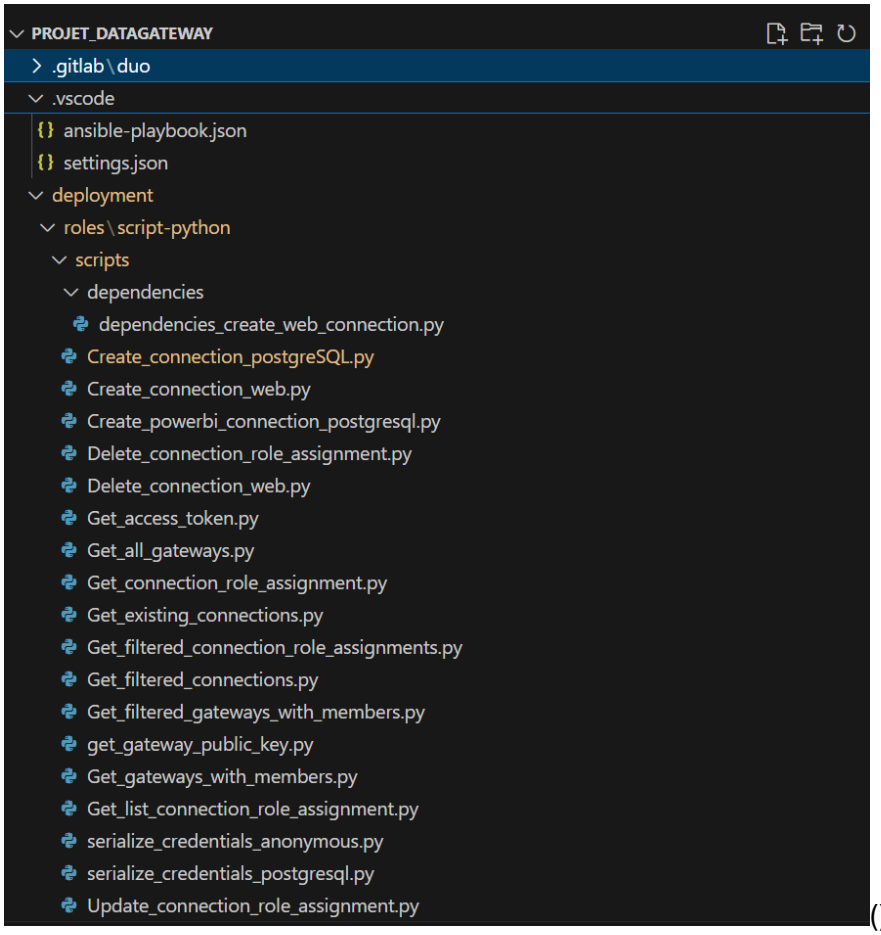
Concernant la validation de mon code, nous ne fonctionnions pas avec un système classique de *Pull Requests*. Ma méthode de travail consistait d'abord à écrire des fonctions Python qui répondaient au besoin et qui étaient fonctionnelles. Ensuite, je revoyais mon code directement avec Hervé pour discuter des optimisations et des améliorations à apporter.

De son côté, Youssef m'appelait aussi régulièrement pour me demander des explications sur le fonctionnement de mes scripts afin de pouvoir bien les utiliser.

Mise en place de l'environnement de développement

Avant de pouvoir coder efficacement, une étape cruciale a été la mise en place de mon environnement de travail et de l'infrastructure du projet. Pour cette partie assez lourde, Youssef m'a beaucoup accompagné à travers plusieurs réunions. Il m'a aidé à :

- Structurer le projet : Ranger mes fichiers de la bonne façon et créer mon environnement virtuel Python (venv) pour isoler mes dépendances.
- Gérer le versioning : Me connecter au GitLab de l'entreprise et créer ma propre branche de développement. C'est sur cette branche que je poussais mon travail, et Youssef venait y récupérer le code dont il avait besoin pour faire ses propres tests.
- Sécuriser les accès : Me connecter à HashiCorp Vault. Cela m'a permis de récupérer les *credentials* (mots de passe et clés d'API) de manière sécurisée, afin de ne jamais les écrire "en dur" dans mon code source.
- Préparer l'intégration continue (CI/CD) : Me connecter à Jenkins et configurer mon environnement de test via Ansible. Concrètement, cela s'est traduit par la création et la modification d'un fichier Jenkinsfile et de fichiers de configuration Ansible en format .yaml.



()

Organisation du projet

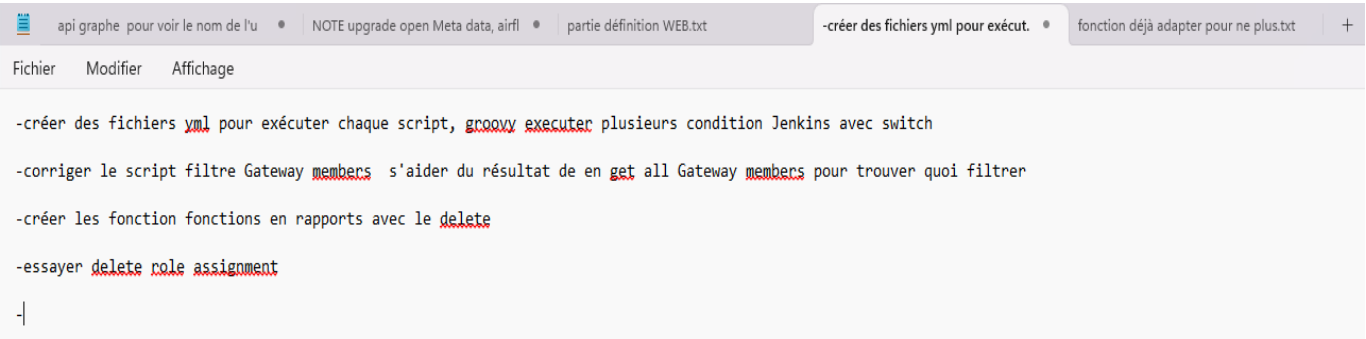
Planning

Pendant la durée de mon stage, j'effectuais des journées de 9h à 17h, du lundi au vendredi. Mon rythme de travail était hybride : j'étais présent sur le site d'EuraTechnologies la majeure partie de la semaine, et je bénéficiais d'un jour de télétravail par semaine. Comme nous ne fonctionnions pas en méthode Agile avec des sprints stricts, l'organisation de mon temps de travail était assez flexible. Elle s'adaptait surtout à l'avancée de mes développements et aux priorités techniques définies avec mon tuteur au fur et à mesure.

Compte-rendu et prise de notes

Tout au long de mon stage, que ce soit pendant mes points avec Hervé ou lorsque je devais assimiler de nouvelles notions techniques, j'ai pris l'habitude de tout noter sur l'application Bloc-notes de mon ordinateur. Ces notes avaient plusieurs objectifs :

- Garder une trace des explications techniques pour bien les comprendre à tête reposée.
- Éviter de poser plusieurs fois les mêmes questions ou de refaire les mêmes erreurs.
- Centraliser au fur et à mesure les informations utiles pour la rédaction de ce rapport de stage.



Réunions

Au-delà des points réguliers avec mon tuteur (une à deux fois par semaine) et des appels techniques avec Youssef, j'étais pleinement intégré à la vie de l'équipe Architecture Solutions (AS). Tous les vendredis matin, de 9h à 10h, je participais à la réunion hebdomadaire de l'équipe. Ce point global permettait à chacun de présenter les informations importantes de la semaine, de discuter des problèmes rencontrés, et de faire remonter les sujets bloquants à l'ensemble du groupe ainsi qu'à Nicolas, le chef de l'équipe. C'était un moment très instructif pour moi car cela me donnait une vision plus large des autres projets gérés par SNCF e.Solutions.

Formation, documentation et premiers jours

Dès mon arrivée, mes premiers jours ont été consacrés à mon intégration technique et administrative. Hervé m'a accompagné pas à pas pour configurer mon poste de travail SNCF : récupération de mes identifiants, mise en place de la double authentification, configuration de ma boîte mail et accès au portail interne.

Ensuite, pour la partie technique de ma mission, j'ai dû m'adapter. Comme je l'ai mentionné plus tôt, la documentation officielle de Microsoft Fabric concernant les API était souvent incomplète. Pour me former et avancer sur mes tâches, je me suis donc très peu appuyé sur des tutoriels classiques. Mes principales sources d'apprentissage ont été :

- L'accompagnement humain, grâce aux explications approfondies d'Hervé et aux conseils d'infrastructure de Youssef.

- La documentation Microsoft.

- L'utilisation ponctuelle d'intelligences artificielles pour m'aider à débloquer des erreurs de code spécifiques en Python ou comprendre certaines configurations.

Réalisation de la mission

Analyse et compréhension de la tâche

Au tout début de ma mission, mon tuteur Hervé m'a fourni un premier script de démonstration rédigé en PowerShell. Ce script était particulièrement complexe : il permettait de créer une connexion Web anonyme sur un cluster Data Gateway, de gérer l'authentification a plusieurs types de connexions, et utilisait des librairies natives .NET (`Microsoft.PowerBI.Api`) pour chiffrer les *credentials* via l'algorithme RSA.

Après avoir étudié ce fonctionnement, nous avons décidé de basculer le développement des futurs scripts en Python. Ce choix s'expliquait par deux raisons : d'une part, j'avais une bien meilleure maîtrise de ce langage, et d'autre part, Python est particulièrement adapté et léger pour interagir de manière native avec des API REST.

La principale difficulté lors de cette phase d'analyse a été la documentation de l'API Microsoft Fabric. Celle-ci était très dispersée et parfois manquante. Avec l'aiguillage d'Hervé, j'ai dû faire beaucoup de recherches. Le plus grand défi a été de trouver et de comprendre la structure exacte des *payloads* (les corps de requêtes JSON) à envoyer, car ceux-ci variaient énormément selon le type de connexion à créer et l'API utilisé (Power BI ou Fabric).

Learn Documentation Formation et Labs Q&A Rubriques

Azure Products Architecture Develop Découvrir Azure Troubleshooting Resources

Certaines parties de cette rubrique peuvent être traduites par une machine ou par l'IA.

Rechercher par titre

- Tâche de copie
- Core
  - Capacités
  - Connections
    - Aperçu
    - Add Connection Role Assignment
    - Create Connection
    - Delete Connection
    - Delete Connection Role Assignment
    - Get Connection
    - Get Connection Role Assignment
    - List Connection Role Assignments
    - List Connections
    - List Supported Connection Types
    - Update Connection
    - Update Connection Role Assignment
  - Pipelines de déploiement
  - Domaines
  - Fournisseur de partages de données externes
  - Destinataire des partages de données externes
  - Dossiers
  - Passerelles
  - Git
  - Objets
  - Planificateur de travaux
  - Opérations de longue durée
  - Points de terminaison privés managés
  - Sécurité de l'accès aux données OneLake
  - Paramètres OneLake
  - Raccourcis OneLake
  - Étiquettes
  - Espaces de travail

Télécharger le PDF

Learn / Fabric REST APIs / Connections / Mode focus

Connections - Create Connection

Service: Core
Version d'API: v1

Crée une connexion.
Pour chiffrer les informations d'identification, consultez Configurer les informations d'identification par programmation.

Permissions

- Si vous créez une connexion de passerelle, l'appelant doit disposer d'autorisations pour la passerelle.
- Les principaux de service doivent disposer d'autorisations accordées par l'administrateur Fabric. Pour plus d'informations, consultez : les principaux de service peuvent créer des espaces de travail, des connexions et des pipelines de déploiement.

Étendues déléguées requises

Connection.ReadWrite.All

Identités prises en charge par Microsoft Entra

Cette API prend en charge les identités Microsoft répertoriées dans cette section.

Agrandir le tableau

Identity	Support
User	Yes
principal de service et identités managées	Yes

Interface

HTTP Copier Essayer

```
POST https://api.fabric.microsoft.com/v1/connections
```

Phase de développement

Pour dialoguer avec les API, je me suis appuyé sur des bibliothèques Python standards et robustes : requests pour effectuer les appels HTTP, json pour formater et traiter les données, os pour la gestion des variables d'environnement, et typing pour typer proprement mon code.

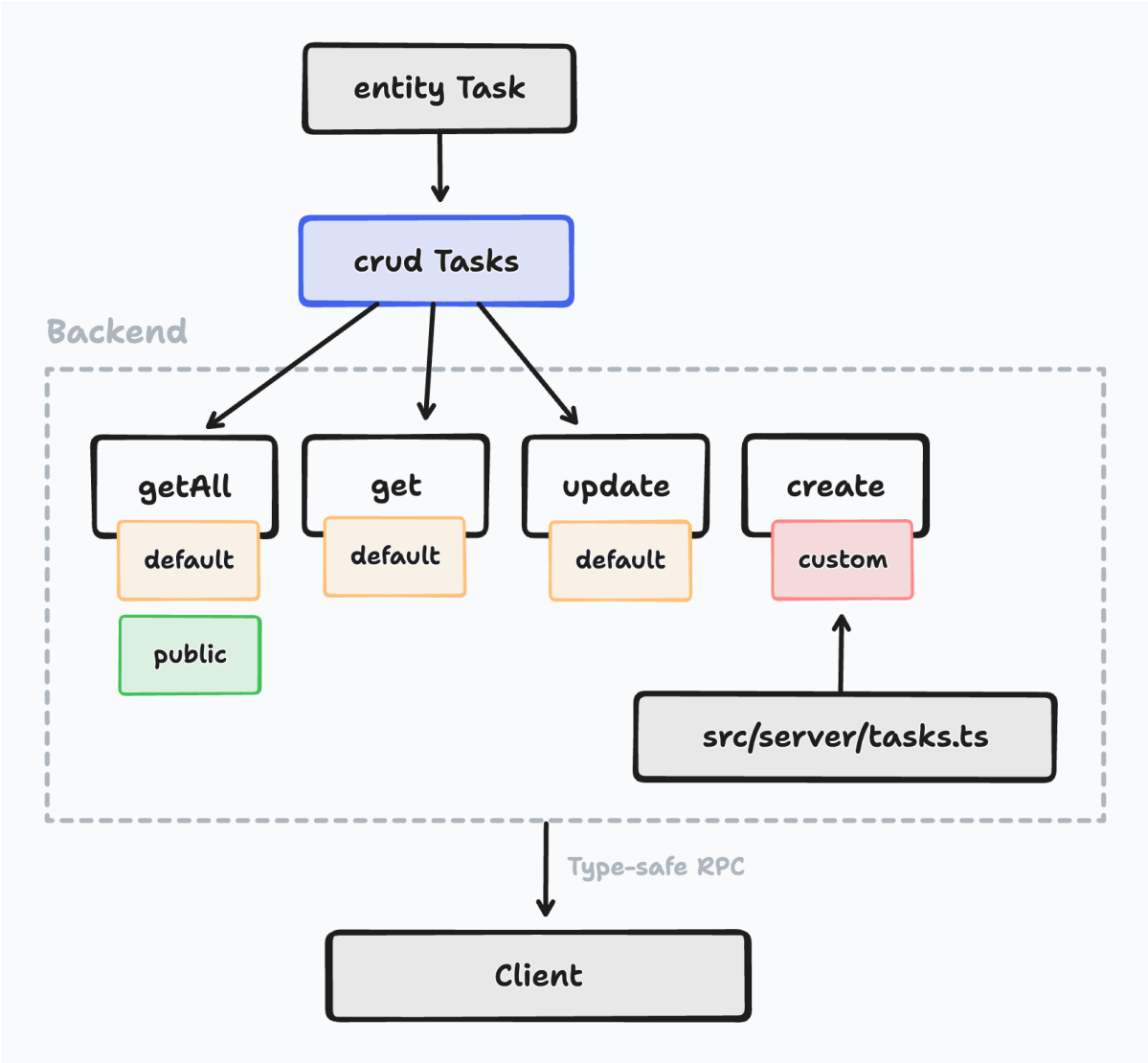
Concrètement, j'ai développé une série de scripts couvrant l'ensemble des opérations CRUD (Create, Read, Update, Delete) nécessaires à la gestion des datagateways.

- Read (Lecture) : La récupération de toutes les passerelles (Get_all_gateways.py), des connexions existantes (Get_existing_connections.py), ou encore la récupération des clés publiques (get_gateway_public_key.py).

- Create (Création) : La création de nouvelles connexions, qu'elles soient de type Web (Create_connection_web.py) ou pour des bases de données (Create_connection_postgreSQL.py).

- Update (Modification) : La mise à jour des paramètres des connexions (Update_connection.py) ou l'attribution des rôles (Update_connection_role_assignment.py).

- Delete (Suppression) : La suppression de connexions (Delete_connection_web.py) et des droits associés.



Un enjeu majeur du développement était la sécurité. Il n'était pas normal de laisser des mots de passe ou des identifiants "en dur" dans le code. Pour gérer l'authentification, mon code Python

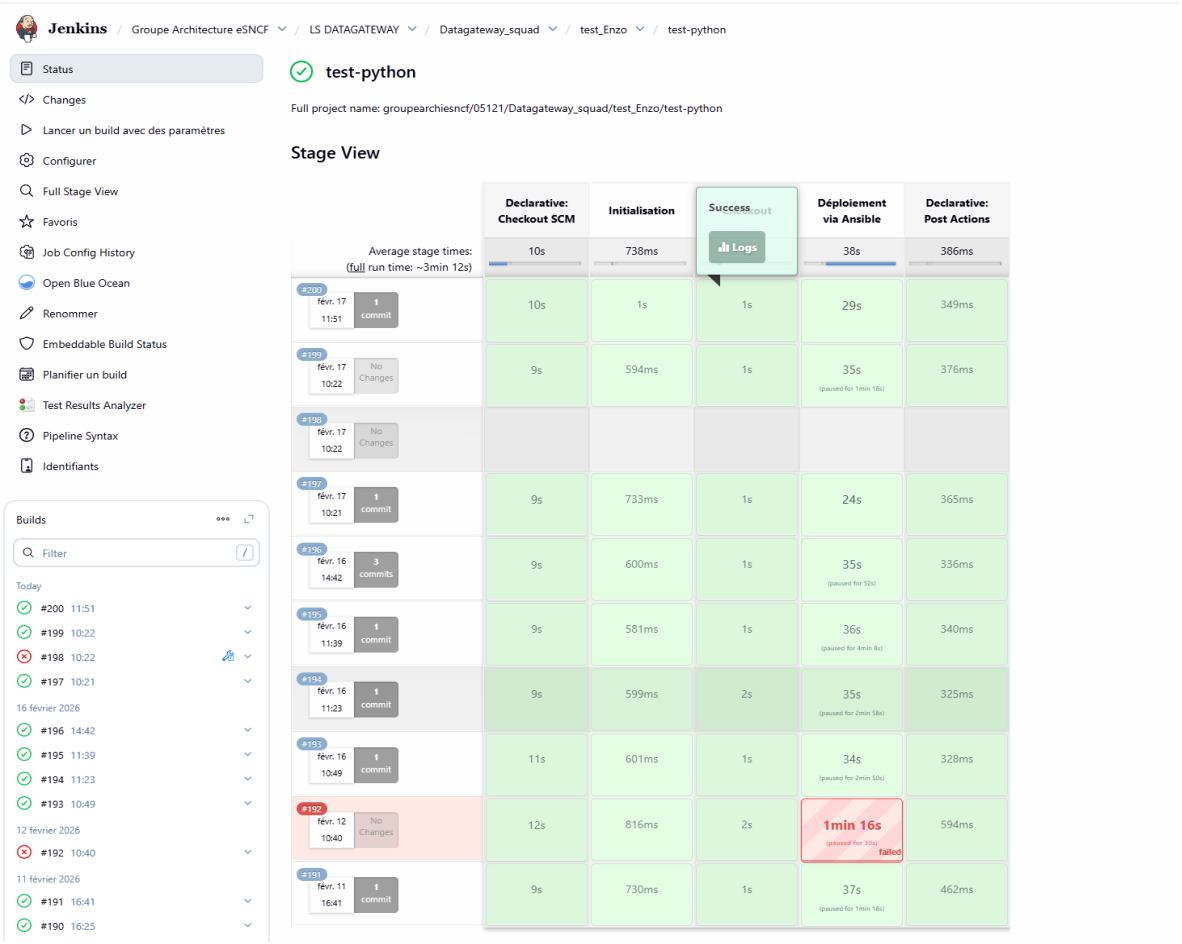
s'interfaçait directement avec HashiCorp Vault afin de récupérer les *tokens* d'accès de manière totalement sécurisée et dynamique.

Intégration et validation

Pour valider le fonctionnement de mon code, je ne me contentais pas de lancer les scripts localement. J'ai utilisé Jenkins pour tester mes développements dans des conditions proches de la production.

Afin de rendre les tests fluides et interactifs, j'ai adapté le fichier de configuration Jenkinsfile. Cela m'a permis d'intégrer des paramètres directement dans l'interface graphique de Jenkins. Par exemple, lorsque je voulais tester la fonction de récupération d'une passerelle, je pouvais sélectionner l'action dans un menu déroulant (comme "Get all gateways" ou "Create PBI PostgreSQL connection") et renseigner l'ID de la gateway directement depuis l'interface Jenkins.

Dans mon code Python, j'utilisais ensuite la bibliothèque os (avec des appels comme os.getenv()) pour récupérer ces valeurs passées par Jenkins. Cette méthode garantissait que les variables n'étaient jamais écrites dans le code source lors des *builds*.



Documentation et restitution du travail

Bien que nous n'utilisions pas de système de tickets rigide, il était indispensable de garder une trace claire de mes développements pour que le reste de l'équipe, et notamment Youssef, puisse réutiliser mon travail.

Pour cela, j'ai documenté mon code en intégrant des *docstrings* (chaînes de documentation) au tout début de chaque fonction Python. J'y expliquais en détail le but de la fonction, les paramètres attendus en entrée (comme les ID de cluster ou les URL), et le type de retour fourni. Cette démarche a grandement facilité les revues de code avec mon tuteur et assuré la maintenabilité du projet sur le long terme.

Justification de la solution retenue

Comparatif logiciel et choix technologiques

Pour l'environnement de développement intégré (IDE), j'ai utilisé Visual Studio Code (VS Code). C'est un outil léger, très répandu dans le monde professionnel, et ses nombreuses extensions (pour Python, Git, etc.) ont grandement facilité la rédaction et la coloration syntaxique de mon code.

Concernant les tests d'API, je n'utilisais pas d'outils externes comme Postman. Je préférais coder directement mes requêtes en Python et faire mes tests via des *playbooks* Ansible, le tout exécuté et orchestré depuis Jenkins. Cela permettait de tester directement le code tel qu'il serait utilisé dans l'environnement cible.

Choix d’organisation et outils de gestion

L'organisation de ce projet ne nécessitait pas d'outils de gestion de tickets lourds comme Jira. Notre suivi reposait sur une communication directe et quotidienne via Microsoft Teams avec mon tuteur Hervé, et sur la prise de notes sur le Bloc-notes de mon poste de travail.

Pour la gestion du code et la collaboration avec Youssef, nous avons eu besoin de GitLab, lorsque j'avais terminé et testé une fonction, Youssef récupérait simplement mon code pour l'intégrer à ses tests personnels.

Étapes essentielles

Le développement de chaque nouveau script Python ne se faisait pas d'une traite. J'ai mis en place une méthodologie itérative de plusieurs étapes pour m'assurer que chaque fonctionnalité était sécurisée :

1. Comprendre la documentation et l'API : Avant même d'écrire une ligne de code, la première étape consistait à identifier la bonne route d'API à appeler. Je devais souvent analyser des exemples partiels ou échanger avec mon tuteur pour trouver les paramètres obligatoires et comprendre l'arborescence des données (le format JSON attendu).

2. Créer la fonction dans VS Code : Une fois la logique d'appel API bien définie, je passais à la phase de développement dans VS Code. Mon objectif était de produire un code propre, sécurisé et réutilisable. La création d'un script complet impliquait plusieurs étapes techniques à l'intérieur même du code :

   L'importation de la sécurité : Je commençais par structurer mon fichier avec les bibliothèques essentielles (requests, json, os). Pour respecter nos normes de sécurité, j'utilisais os.getenv() afin de récupérer dynamiquement les *credentials* (comme le CLIENT_ID ou les mots de passe) depuis HashiCorp Vault. Ainsi, aucun identifiant n'était jamais écrit "en dur" dans mon code.

```
tenantid = os.getenv("tenantid")
clientid = os.getenv("clientid")
clientsecret = os.getenv("clientsecret")
GATEWAY_ID = os.getenv("GATEWAY_ID", "")
CONNECTION_TYPE = os.getenv("CONNECTION_TYPE", "").strip()
AUTH_TYPE = os.getenv("AUTH_TYPE", "").strip()
CONNECTION_NAME = os.getenv("CONNECTION_NAME", "ASO-Connection")
PRIVACY_LEVEL = os.getenv("PRIVACY_LEVEL", "Organizational")
```

   La documentation intégrée (*Docstrings*) : J'ajoutais systématiquement des chaînes de documentation au début de chaque fonction pour expliquer clairement son rôle, les paramètres attendus en entrée et le résultat retourné, facilitant ainsi la relecture par

l'équipe.

```python
def get_all_gateways_fabric(access_token: str, page_size: int = 100, max_pages: int | None = None) -> list[dict]:
    """Récupère toutes les gateways Fabric en gérant la pagination.

    - Utilise $top pour limiter le nombre d'éléments par page.
    - Suit soit un "continuationToken", soit un "@odata.nextLink" si présent
      (cela dépend de la façon dont l'API Fabric renvoie la pagination).

    Args:
        access_token: jeton d'accès.
        page_size: nombre d'éléments par page (par ex. 100).
        max_pages: limite de pages à charger (None = pas de limite explicite).

    Returns:
        list[dict]: liste complète des gateways trouvées.
    """
    print("\n>>> get_all_gateways_fabric()")
```

La gestion des appels préalables **:** Souvent, la requête finale nécessitait des données préliminaires. Par exemple, avant de créer une connexion, je devais coder une première requête pour interroger l'API Fabric afin d'obtenir la liste des membres (les nœuds) du cluster ciblé et extraire leurs clés publiques (*modulus* et *exponent*).

Le chiffrement des données sensibles : C'était l'une des phases les plus techniques. La consigne de Microsoft étant stricte, je devais formater les mots de passe et noms d'utilisateur au format JSON, puis appliquer une fonction de chiffrement asymétrique (en utilisant les clés publiques récupérées juste avant). Le but était d'obtenir une chaîne de *credentials* totalement cryptée.

La construction minutieuse du *Payload* : Une fois les données chiffrées, je devais assembler le corps de la requête finale (le *payload*). Cela passait par la création de dictionnaires Python complexes et imbriqués.
Je lançais l'appel HTTP (via requests.post, get, patch ou delete) en intégrant le *token* d'authentification Fabric dans les *headers*. J'intégrais systématiquement des méthodes comme response.raise_for_status() et des blocs try/except pour capturer proprement les erreurs HTTP. Cela permettait d'afficher des messages d'erreur clairs (souvent renvoyés au format JSON par l'API), ce qui nous faisait gagner un temps précieux lors du débuggage avec Hervé.

```
payload = {
    "connectivityType": "OnPremisesGateway",
    "gatewayId": GATEWAY_ID,
    "displayName": CONNECTION_NAME,
    "connectionDetails": {
        "type": "PostgreSQL",
        "creationMethod": "PostgreSql",
        "parameters": [
            {"dataType": "Text", "name": "server", "value": SERVER_NAME},
            {"dataType": "Text", "name": "database", "value": DATABASE_NAME},
        ],
    },
    "privacyLevel": PRIVACY_LEVEL,
    "credentialDetails": {
        "singleSignOnType": "None",
        "connectionEncryption": "Encrypted",
        "skipTestConnection": SKIPTESTCONNECTION,
        "credentials": {
            "credentialType": "Basic",
            "values": credential_values,
        },
    },
}
```

3. **Pousser sur GitLab :** Une fois le script fonctionnel en local, je *commitais(git commit)* et je poussais (git *push*) mon code sur ma branche(test-scripts) de développement via Git. Cela permettait de versionner mon travail et de le rendre accessible au reste de l'équipe, notamment à Youssef pour ses propres tests d'infrastructure.

4. **Tester et déployer via Jenkins :** C'était l'épreuve de vérité. J'exécutais mon code via un pipeline Jenkins qui déclenchait des *playbooks* Ansible. Grâce au fichier Jenkinsfile que j'avais paramétré, je pouvais injecter dynamiquement les variables (comme l'ID de la gateway à cibler) directement depuis l'interface graphique de Jenkins, validant ainsi le fonctionnement du script dans un environnement iso-production.

Fonctionnalités réalisées

Parmi l'ensemble des scripts de gestion (CRUD) que j'ai réalisés, le développement du script Create_connection_postgreSQL.py a été l'un des plus formateurs et des plus complexes techniquement. Il illustre parfaitement la logique d'automatisation que nous voulions mettre en place.

L'objectif de ce script est de provisionner automatiquement une nouvelle connexion à une base de données PostgreSQL sur un cluster de datagateways. Voici comment j'ai structuré la logique du code pas à pas :

- **Authentification préalable :** Le script débute par l'appel à une fonction dédiée (Get_access_token) qui utilise le CLIENT_ID et le CLIENT_SECRET (récupérés de manière sécurisée) pour obtenir un *token* d'autorisation temporaire auprès de Microsoft Fabric.
- **Récupération de la topologie du cluster :** Un cluster de datagateways est souvent composé de plusieurs machines (ou nœuds) pour assurer la haute disponibilité. Mon script interroge d'abord l'API pour lister tous les membres actifs du cluster visé et récupérer leurs informations techniques.
- **Chiffrement asymétrique :** Pour des raisons évidentes de sécurité, Microsoft interdit d'envoyer les identifiants de la base de données (nom d'utilisateur et mot de passe) en texte brut. Pour chaque nœud du cluster, mon script va extraire sa clé publique (composée d'un *modulus* et d'un *exponent*). Je formate ensuite les identifiants sous forme de chaîne JSON, et j'utilise une fonction de chiffrement asymétrique (AsymmetricKeyEncryptor) pour verrouiller ces données avec la clé publique du nœud.
- **Construction du Payload :** Je rassemble ensuite toutes ces informations dans un grand dictionnaire Python (*payload*). Ce paquet de données est très structuré : il déclare le type de connectivité (OnPremisesGateway), les détails du serveur (PostgreSQL, nom du serveur, nom de la base), et intègre le tableau contenant les mots de passe chiffrés pour chaque nœud du cluster.
- **Exécution de la requête POST :** Enfin, j'envoie ce *payload* à l'API Fabric (/v1/connections). Le script analyse la réponse HTTP de Microsoft : si tout est correct, la connexion est créée instantanément sur le portail et le script nous retourne l'identifiant unique (ID) de la nouvelle connexion.

L'automatisation de cette tâche précise est un vrai gain pour l'équipe : elle supprime des dizaines de clics fastidieux dans les interfaces Microsoft, garantit que les règles de sécurité (chiffrement) sont appliquées sans faille humaine, et permet de déployer des accès aux données en quelques secondes.

Mise en place : outils et incidents

La mise en place de ces solutions n'a pas été sans obstacles. Le défi majeur, tout au long du stage, a été le manque cruel de documentation claire sur les API Microsoft Fabric. Trouver la structure exacte des *payloads* selon le type de connexion (Web, PostgreSQL, etc.) relevait souvent du casse-tête.

À plusieurs reprises, je me suis retrouvé face à des erreurs ou des blocages que j'aurais pu facilement contourner en utilisant l'API Power BI (qui est mieux documentée car elle est utilisée pour executer des requetes CRUD dans powerBI). Cependant, la consigne stricte de mon périmètre était de n'utiliser exclusivement que l'API Fabric.

Même en cherchant dans la documentation disponible, il m'est arrivé d'être complètement bloqué. Heureusement, grâce à son expertise, mon tuteur Hervé savait exactement où chercher. Ensemble, en analysant les retours d'erreurs de l'API, nous avons toujours réussi à débugger les scripts et à trouver la solution adéquate pour faire fonctionner mes développements.

Évaluation des réalisations

Adéquation du travail

À l'issue de ces deux mois de stage, je peux affirmer que les missions qui m'avaient été confiées par mon tuteur ont été remplie à presque 100%, même si le projet global est encore en cours de développement. Le portail final n'est pas encore déployé, mais j'ai pu réaliser une très bonne partie du travail de fond en posant les bases techniques (les scripts Python) nécessaires à son futur fonctionnement.

D'un point de vue personnel, ce stage a parfaitement répondu à mes attentes en matière de montée en compétences. J'ai pu consolider mes connaissances préalables sur les appels d'API, qui étaient ici d'un niveau de complexité supérieur. Surtout, j'ai découvert et appris à manipuler des outils d'infrastructure et d'automatisation majeurs que je ne connaissais pas, tels que Jenkins et Ansible. C'est d'ailleurs avec la forte envie de poursuivre et d'achever ce projet que j'espère pouvoir continuer l'aventure chez SNCF e.Solutions l'année prochaine dans le cadre d'une alternance.

Réactions des demandeurs et intérêt pour l'entreprise

Les retours au quotidien de la part d'Hervé et de Youssef ont été positifs. Le travail demandé a été fourni en temps et en heure, et les scripts développés s'intègrent bien dans l'architecture prévue.

L'intérêt de mes réalisations pour l'équipe AS et l'entreprise se mesurera pleinement lors de la sortie du portail Automatio. Concrètement, mes scripts constituent le "moteur" caché de ce futur portail. Une fois l'interface graphique branchée sur mon code, l'entreprise y gagnera sur deux tableaux majeurs :

- **Un gain de temps considérable :** Les équipes n'auront plus à chercher dans les consoles Microsoft ou à taper des lignes de commande pour gérer les datagateways.
- **Une sécurité renforcée :** L'automatisation supprime le risque d'erreur humaine (faute de frappe, mauvaise configuration) lors de la création ou de la modification des passerelles en production.

Propositions d’amélioration

Si mon stage avait duré un mois supplémentaire, ma priorité absolue aurait été de finaliser le développement de l'intégralité des fonctions CRUD demandées pour couvrir 100 % des cas d'usage de l'API Fabric.

Pour la suite du projet, ou pour la personne qui reprendra mes travaux, la principale piste d'amélioration et la prochaine grande étape consisteront à relier directement ces scripts Python (actuellement exécutables via Jenkins) à l'interface graphique utilisateur du portail Automatio. Cela permettra d'aboutir au produit fini : un portail où la création complexe d'une passerelle sécurisée se résumera à un simple clic sur un bouton.

La démarche RSE de SNCF e.Solutions

SNCF e.Solutions, en tant que bras technologique de SNCF Connect & Tech, place la Responsabilité Sociétale des Entreprises (RSE) au cœur de sa stratégie d'innovation. En tant qu'acteur majeur du numérique et des mobilités partagées, l'entreprise a pleinement conscience de son rôle dans la transition écologique et sociétale. Sa démarche vise à concilier performance technologique et impact positif pour la société.

Engagement social

L'entreprise accorde une grande importance à l'inclusion, à la diversité et au bien-être de ses collaborateurs. SNCF Connect & Tech s'engage activement pour l'égalité femmes-hommes, notamment via le réseau "SNCF Mixité", et propose un accompagnement spécifique pour l'intégration des travailleurs en situation de handicap. De plus, l'entreprise mène une politique très volontariste pour l'insertion professionnelle des jeunes étudiants. En tant que stagiaire, j'ai pu bénéficier d'un encadrement bienveillant (notamment grâce au compagnonnage technique avec mon tuteur Herve chapuis), ce qui a grandement favorisé mon intégration et ma montée en compétences.

Engagement environnemental

Dans le secteur de la tech, l'impact écologique des infrastructures est un défi majeur. C'est pourquoi SNCF e.Solutions déploie une stratégie forte autour du "Numérique Responsable" (Green IT). L'entreprise réalise des bilans réguliers de ses émissions de gaz à effet de serre afin de mettre en place des trajectoires de réduction de son empreinte carbone. En interne, les équipes sont formées et sensibilisées à l'éco-conception des services numériques, et des travaux sont menés pour mesurer et limiter l'impact écologique des infrastructures Cloud et de l'Intelligence Artificielle. À l'échelle des locaux, des partenariats sont également en place pour le recyclage des déchets de bureau.

Engagement économique local

Avec son implantation au sein d'EuraTechnologies à Lille, SNCF e.Solutions s'ancre profondément dans un écosystème technologique de premier plan. Ce choix géographique permet à l'entreprise de participer au dynamisme économique de la région Hauts-de-France, de collaborer avec des talents locaux et de rayonner au sein du plus grand pôle d'excellence numérique d'Europe. Par ses activités, l'entité contribue directement à la modernisation des outils de gestion de la donnée, bénéficiant indirectement à des millions de voyageurs sur le territoire.

Éthique professionnelle

La culture d'entreprise repose sur des valeurs fondamentales telles que la Performance, l'Ouverture et la Confiance. Ces principes se traduisent au quotidien par un environnement de travail où la transparence, la communication (via Teams ou nos réunions hebdomadaires ASO) et l'entraide sont primordiales. Bien que notre équipe n'utilise pas la méthode Agile classique de manière stricte, l'organisation du travail reste profondément collaborative et humaine, favorisant l'autonomie et la responsabilité individuelle.

Conclusion

Ce stage de deux mois au sein de SNCF e.Solutions, au cœur de l'écosystème d'EuraTechnologies à Lille, a marqué une étape décisive dans mon parcours de formation en informatique. Si ma précédente expérience m'avait permis de découvrir le fonctionnement d'une Entreprise de Services du Numérique (ESN) à taille humaine et de me familiariser avec le développement web classique, cette immersion dans un grand compte industriel répondait à un tout autre objectif

Un bond en avant technique et méthodologique

Sur le plan technique, la mission qui m'a été confiée au sein de l'équipe Architecture Solutions (AS) m'a permis de sortir de ma zone de confort. Le projet DataGateway m'a propulsé dans l'univers de l'automatisation, du DevOps et de l'architecture Cloud, des domaines que je n'avais jusqu'alors que peu abordés. J'ai eu l'opportunité de passer de la théorie à la pratique en concevant de A à Z une suite de scripts Python capables d'interagir nativement avec les API Microsoft Fabric.

Ce développement m'a poussé à approfondir mes connaissances sur les requêtes HTTP (CRUD), mais surtout à intégrer des notions de sécurité primordiales en entreprise. La manipulation d'outils comme HashiCorp Vault pour la gestion dynamique des *credentials* et la mise en place de chiffrements asymétriques (RSA) pour la transmission de mots de passe m'ont fait prendre conscience des normes de sécurité strictes régissant les données d'un groupe comme la SNCF. Par ailleurs, l'intégration de mon code dans des pipelines d'intégration continue (CI/CD) via Jenkins et l'utilisation d'Ansible pour orchestrer mes tests m'ont initié aux standards actuels de l'industrialisation logicielle.

Une intégration humaine réussie

Au-delà des lignes de code, je retiendrai de cette expérience la qualité de mon intégration humaine. Intégrer un grand groupe peut parfois sembler intimidant, mais l'accueil chaleureux de l'équipe AS a grandement facilité mon adaptation. Ma participation active aux réunions hebdomadaires du vendredi matin m'a donné une vision globale des enjeux du service, me permettant de comprendre comment mon travail s'inscrivait dans la stratégie globale du département dirigé par Nicolas. La collaboration fluide avec Youssef, notamment sur la mise en place de mon environnement de développement sur GitLab et Jenkins, m'a également prouvé l'importance de la communication inter-équipes pour le succès d'un projet d'infrastructure.

Perspectives et projet professionnel

En définitive, j'ai le sentiment d'avoir pleinement accompli la mission qui m'était confiée pour cette phase initiale du projet. Bien que le portail Automatio final ne soit pas encore déployé, les fondations techniques (le "moteur" Python) sont désormais opérationnelles pour les principales opérations de gestion des *datagateways*. Savoir que mon code contribuera à terme à faire gagner

du temps aux équipes internes et à sécuriser les flux de données de la SNCF est une réelle source de satisfaction.

Cette immersion a non seulement confirmé mon attrait pour les métiers de l'infrastructure, du DevOps et de la donnée, mais elle a aussi conforté mon souhait de poursuivre mon évolution au sein de ce type d'environnement. C'est pourquoi j'ai la forte volonté de prolonger mon engagement chez SNCF e.Solutions l'année prochaine, dans le cadre d'une alternance. Cela me permettrait non seulement de finaliser les cas d'usages restants sur les API Fabric, mais surtout de participer à la prochaine grande étape du projet : le raccordement de mes scripts à l'interface graphique utilisateur finale.